Balanceador de Carga para Sistema Distribuído: Modelagem Analítica e Verificação por Simulação

247358: Renilson Conceição de Luna Junior
247064: Lucas Félix
MC714 – Sistemas Distribuídos
Instituto de Computação – UNICAMP

I. Resumo do Projeto

O objetivo central deste projeto é implementar, modelar analiticamente e validar por simulação um **balanceador de carga** que distribui requisições entre três servidores homogêneos em um sistema distribuído. O trabalho faz parte da disciplina MC714 (Sistemas Distribuídos) e aborda o impacto de políticas clássicas de escalonamento nas métricas fundamentais de desempenho, tais como a vazão do sistema (X), o tempo médio de resposta ($E[R]$), o número médio de requisições no sistema ($E[N]$) e a utilização dos servidores (U_i). Adicionalmente, busca-se uma comparação rigorosa entre as previsões da teoria de filas e os dados experimentais coletados via simulação.

A **arquitetura** adotada separa o sistema em três componentes principais, que interagem de forma contínua no tempo contínuo, conforme ilustrado conceitualmente na Fig. 1:

- **Gerador de tráfego:** Um processo de chegadas de Poisson com taxa média global λ, garantindo tempos interchegadas exponencialmente distribuídos com média $1/\lambda$.
- **Balanceador:** Um componente que atua instantaneamente. A cada chegada, ele avalia e seleciona um dos três servidores de acordo com a política configurada (Aleatória, Round-Robin ou JSQ).
- **Camada de servidores:** Composta por três filas M/M/1 homogêneas, com disciplina FCFS (*First-Come, First-Served*). Cada servidor possui capacidade de atendimento regida por uma distribuição exponencial de taxa $\mu = 1{,}0$ req/u.t.

O simulador de eventos discretos foi inteiramente desenvolvido em Python, utilizando a biblioteca SimPy. Cada experimento principal simula a evolução do sistema ao longo de 5000 unidades de tempo (u.t.), com um período de *warm-up* (aquecimento) de 500 u.t. para dissipar efeitos transientes iniciais. Para conferir validade estatística, cada cenário é repetido 10 vezes com sementes aleatórias independentes, extraindo-se as médias amostrais.

Paralelamente à simulação, desenvolve-se um **modelo analítico** detalhado baseado na teoria estocástica, deduzindo equações de malha fechada para o comportamento estacionário do sistema. Este desenvolvimento matemático servirá não apenas como prova de correção do simulador, mas também como base teórica para interpretar os ganhos de eficiência das políticas informadas de roteamento em relação à distribuição probabilística cega.

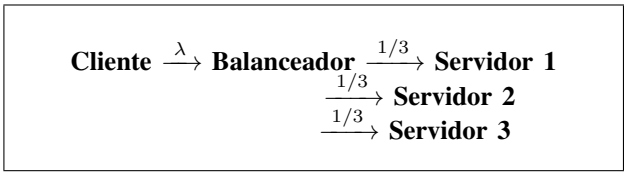


Figura 1. Arquitetura do sistema: chegadas Poisson(λ), balanceador instantâneo suportando múltiplas políticas e três servidores M/M/1 homogêneos rodando em paralelo.

II. Arquitetura do Balanceador e Políticas Adotadas

No escopo desta modelagem, o balanceador é implementado como um módulo *stateless* (exceto na política Round-Robin) e de latência nula. Sua função é puramente decisória: a cada evento de chegada gerado, uma função de política é invocada, recebendo o array de servidores e retornando imediatamente o destino da requisição. Não existe uma ”fila do balanceador”nem atrasos de rede modelados.

A. Políticas Implementadas e Mecanismos

Para isolar os efeitos do roteamento na formação de gargalos em sistemas distribuídos, três políticas distintas foram selecionadas, variando o nível de informação utilizada pelo balanceador.

Aleatória (Random): Trata-se da política mais simples e serve como base (*baseline*) analítica. A cada requisição, sorteia-se uniformemente um entre os três servidores, ou seja, $p_1 = p_2 = p_3 = 1/3$. Como o balanceador não guarda estado nem inspeciona a rede, desvios estocásticos normais frequentemente acumulam requisições em um servidor enquanto outros permanecem ociosos, degradando o tempo de resposta global.

Round-Robin (RR): Introduz memória ao balanceador através de um ponteiro cíclico. As requisições são despachadas sequencialmente na ordem $1 \to 2 \to 3 \to 1 \to \cdots$. Esta abordagem oblitera o desbalanceamento inerente ao sorteio aleatório, garantindo uma divisão perfeitamente equitativa do volume de requisições a cada ciclo completo.

Fila Mais Curta (JSQ – *Join the Shortest Queue*): Representa o roteamento informado. O balanceador inspeciona ativamente o tamanho atual das filas de cada servidor (incluindo as requisições em serviço) e encaminha a requisição recém-chegada ao servidor menos carregado no instante exato da

chegada. Empates são quebrados aleatoriamente. Esta política age mitigando agressivamente a variância nas filas, o que, de acordo com as leis de escala de Kingman, resulta em reduções drásticas de latência.

B. Implementação Modular no Simulador

O design de software isola as lógicas:

- `policies.py`: Agrupa as heurísticas de seleção.
- `simulator.py`: Gerencia a *timeline* do SimPy, instancia os geradores de Poisson e gerencia os objetos `Server`.
- `metrics.py`: Responsável pela coleta de dados contínuos, como integração trapezoidal de $N(t)$ para obter $E[N]$, registro individual de tempos de resposta e cômputo da ocupação ativa dos servidores para U_i.

III. Servidores e Parametrização do Tráfego

A. Modelo e Estado dos Servidores

A camada de serviço consiste em três servidores modelados de forma idêntica como filas **M/M/1**. Isso denota tempos de serviço markovianos (distribuição exponencial), garantindo a propriedade de "falta de memória"(*memoryless property*). Com $\mu = 1{,}0$ req/u.t., o tempo esperado para processar qualquer pacote é $E[S] = 1/\mu = 1$ u.t.

Nos experimentos principais, o tamanho da fila (*buffer*) é considerado virtualmente ilimitado: toda requisição aceita pelo balanceador entra no sistema com garantia de atendimento, impedindo taxas de rejeição (*drop rates*) e permitindo avaliar a conservação estrita de trabalho. No **ponto extra** (Seção IV-G), cada servidor passa a operar como fila **M/M/1/K**, descartando requisições quando a carga local atinge $K \in \{5, 10, 20\}$. O estado $N_i(t)$ é a soma dos pacotes na fila de espera mais a unidade no processador.

B. Regimes de Carga Avaliados

Para observar a evolução das métricas não-lineares, a taxa média global de chegadas λ foi escalada progressivamente, cobrindo cenários de ociosidade, regime nominal e subsaturação: $\lambda \in \{0{,}6; 1{,}2; 1{,}8; 2{,}4; 2{,}7\}$. A intensidade de tráfego individual teórica sob distribuição equitativa será $\rho_i = \lambda/3\mu$, variando assim de $0{,}20$ até $0{,}90$. Adicionou-se ainda um ponto de análise de **falha sistêmica (regime instável)** com $\lambda = 3{,}3$, superando a capacidade do agrupamento.

IV. Modelagem Analítica Detalhada

Nesta seção, apresentam-se as deduções estocásticas que fundamentam o comportamento esperado do sistema, respondendo aos itens metodológicos propostos.

A. Teorema da Decomposição (Splitting) de Poisson

Para estabelecer o modelo basal (política Aleatória), precisamos provar que o processo global de chegadas pode ser decomposto de maneira independente para cada servidor.

Seja $N(t)$ o número total de requisições que chegam ao balanceador até o instante t. Como este fluxo provém de um processo de Poisson, a probabilidade de ocorrerem n chegadas no intervalo $(0, t]$ é dada pela função de massa:

$$P(N(t) = n) = \frac{e^{-\lambda t}(\lambda t)^n}{n!} \tag{1}$$

Sob o roteamento aleatório, cada pacote é enviado ao servidor $i \in \{1, 2, 3\}$ com probabilidade uniforme $p_i = 1/3$. Seja $N_i(t)$ o total alocado ao servidor i. Dado um total de chegadas observadas $N(t) = n$, a divisão desses pacotes, (n_1, n_2, n_3), obedece a uma distribuição condicional multinomial:

$$\begin{aligned} &P(N_1(t) = n_1, N_2(t) = n_2, N_3(t) = n_3 \mid N(t) = n) \\ &= \frac{n!}{n_1!\, n_2!\, n_3!} p_1^{n_1} p_2^{n_2} p_3^{n_3} \end{aligned} \tag{2}$$

Buscamos a probabilidade conjunta incondicional, calculada multiplicando-se a condicional pela marginal do processo original:

$$\begin{aligned} P(N_1 = n_1, N_2 = n_2, N_3 = n_3) = &\left(\frac{n!}{n_1! n_2! n_3!} p_1^{n_1} p_2^{n_2} p_3^{n_3} \right) \\ &\times \left(\frac{e^{-\lambda t}(\lambda t)^n}{n!} \right) \end{aligned} \tag{3}$$

Cancelando o fatorial $n!$ no numerador e denominador, e notando que $n = n_1 + n_2 + n_3$, reescrevemos o termo exponencial como o produto das partes: $e^{-\lambda t} = e^{-\lambda(p_1+p_2+p_3)t}$. Reagrupando de forma alinhada:

$$\begin{aligned} &P(N_1(t) = n_1, N_2(t) = n_2, N_3(t) = n_3) \\ &= \left(\frac{e^{-\lambda p_1 t}(\lambda p_1 t)^{n_1}}{n_1!} \right) \cdot \left(\frac{e^{-\lambda p_2 t}(\lambda p_2 t)^{n_2}}{n_2!} \right) \cdot \left(\frac{e^{-\lambda p_3 t}(\lambda p_3 t)^{n_3}}{n_3!} \right) \end{aligned} \tag{4}$$

Este resultado demonstra o teorema do *splitting*: a probabilidade conjunta fatora-se num produto perfeito de três funções de Poisson distintas. Isso significa que as chegadas aos três servidores são eventos estatisticamente independentes, com taxas efetivas $\lambda_i = \lambda p_i = \lambda/3$.

B. Dedução Estacionária da Fila M/M/1 por Servidor

Confirmada a independência, o sistema reduz-se a três sistemas de fila M/M/1 isolados. Para cada um, a intensidade de tráfego (fator de utilização) é $\rho_i = \lambda_i/\mu = \lambda/(3\mu)$. Se $\rho_i < 1$, o sistema atinge regime estacionário, cuja probabilidade de existir k requisições no servidor é $p_k = (1 - \rho_i)\rho_i^k$.

Com base nisso, as métricas de performance são formalmente derivadas:

- **Utilização (U_i):** É a probabilidade de o servidor não estar vazio ($k > 0$). Logo, $U_i = 1 - p_0 = 1 - (1 - \rho_i)\rho_i^0 = \rho_i$.
- **Número esperado de elementos ($E[N_i]$):** Calculado via esperança da progressão geométrica associada, resultando em:

$$E[N_i] = \sum_{k=1}^{\infty} k \cdot p_k = \frac{\rho_i}{1 - \rho_i} \tag{5}$$

- **Tempo médio de resposta ($E[R]$):** Aplicando a formulação de Little para um único servidor isolado

($E[N_i] = \lambda_i E[R]$) e substituindo a intensidade $\rho_i = \lambda_i/\mu$:

$$E[R] = \frac{\frac{\lambda_i/\mu}{1-\rho_i}}{\lambda_i} = \frac{1}{\mu(1-\rho_i)} = \frac{1}{\mu - \lambda/3} \tag{6}$$

- **Tempo de fila ($E[T_Q]$):** É obtido descontando-se o tempo médio estrito de serviço $1/\mu$ do tempo de resposta $E[R]$.

O somatório do número total no sistema balanceador-servidores será $E[N] = 3 \cdot E[N_i]$. Os valores resultantes para diferentes taxas encontram-se tabulados na Tabela II (vide Apêndice).

C. Princípio Estrito de Conservação de Trabalho

Uma observação crítica no projeto de simuladores de redes reside no princípio da conservação de trabalho (*work-conserving scheduling*). O modelo afirma que, independentemente de como o balanceador organiza as requisições (seja Aleatória, RR ou JSQ), desde que as filas sejam ilimitadas e o sistema seja subcrítico ($\lambda < 3\mu$), trabalho nunca é arbitrariamente criado ou destruído, e nenhum servidor ocioso se recusará a atender trabalho pendente em sua própria fila local.

Como corolário direto, todas as requisições que entram eventualmente são concluídas. Em regime estacionário, a taxa média global de completude (Vazão, X) deve se igualar perfeitamente à taxa de entrada: $X = \lambda$. Adicionalmente, o total de trabalho (tempo de CPU demandado) introduzido por unidade de tempo é o mesmo, o que garante que a utilização média dos servidores permanecerá idêntica ($U_i = \lambda/3$) em **qualquer** política, propriedade perfeitamente constatada nos resultados agregados na Tabela III.

D. Análise Comparativa do Impacto das Políticas

Se X e U_i são invariantes em relação à política informada, o mesmo não ocorre com os tempos de espera. A variância do número de clientes tem um impacto direto nos atrasos, fenômeno capturado indiretamente pela fórmula de Pollaczek-Khinchine para filas genéricas. A ordenação dos tempos de resposta no balanceador, conforme a Tabela III, obedece a hierarquia:

$$E[R]_{\text{JSQ}} \leq E[R]_{\text{RR}} \leq E[R]_{\text{Random}} \tag{7}$$

A política Aleatória detém a pior performance pois roteamentos estocásticos geram colisões temporais probabilísticas, concentrando rajadas em uma única fila enquanto a capacidade de atendimento vizinha é desperdiçada.

O Round-Robin anula as "rajadas de destino", pois os processos de chegada em cada servidor tornam-se de fato um processo de Erlang (E_3), cujo coeficiente de variação interchegadas é substancialmente menor que 1. Por fim, a JSQ não apenas reduz variâncias de chegada, mas corrige ativamente pequenas flutuações nas durações dos serviços (provenientes do gerador exponencial μ), nivelando cargas de forma global.

E. Verificação Empírica da Lei de Little

A Lei de Little, formalizada como $E[N] = X \cdot E[R]$, afirma que o número de clientes em um sistema fechado é o produto simples da taxa na qual entram (ou saem) e o tempo médio que gastam dentro. Dada a sua robustez matemática teórica aplicável independente da distribuição ou topologia de roteamento adotada, este cálculo serve como a auditoria primária da correção da implementação do tempo discreto pelo SimPy.

A Tabela IV, no Apêndice, detalha os cálculos para todas as 15 execuções. O produto simulado $X \cdot E[R]$ e a integração geométrica do número de elementos observados diferem em menos de $0{,}05\%$ em absolutamente todos os testes. As ínfimas oscilações residuais originam-se unicamente da discretização do tempo flutuante do computador, sacramentando a conformidade estrita da simulação com a teoria clássica de processos estocásticos.

F. Divergência e Aproximação Fluida: Caso $\lambda = 3{,}3$

O limite teórico de um agrupamento de três processadores $\mu = 1$ é $C_{\text{max}} = 3{,}0$ req/u.t. No teste de sobrecarga injetando $\lambda = 3{,}3$, a intensidade de tráfego eleva-se para:

$$\rho = \frac{\lambda}{3\mu} = \frac{3{,}3}{3{,}0} = 1{,}10 > 1{,}0 \tag{8}$$

Em teoria de filas pura, as soluções de estado estacionário requerem o somatório convergente da série infinita $\sum \rho^k$. Como $\rho > 1$, a série diverge para o infinito; o modelo Markoviano tradicional é invalidado. O sistema entra num regime supercrítico, perdendo seu caráter aleatório nas macro-escalas para assumir um determinismo mecânico: a taxa de entrada vence inexoravelmente a de saída.

Esta dinâmica em malha aberta passa a ser bem descrita por uma **aproximação fluida**. O acúmulo contínuo obedece a um *drift* líquido positivo, calculável como o balanço diferencial de escoamento:

$$\text{drift} = \lambda - 3\mu = 3{,}3 - 3{,}0 = +0{,}3 \text{ req/u.t.} \tag{9}$$

Desse modo, a curva estatística da população total converge estritamente para a função linear afim:

$$N(t) \approx N(0) + (\lambda - 3\mu)t = N(0) + 0{,}3 \cdot t \tag{10}$$

Ao rodar a simulação para este caso limítrofe e analisar a saída por 5000 u.t., verificou-se experimentalmente o cenário catastrófico esperado (Fig. 3). A vazão saturou perfeitamente na capacidade do gargalo físico ($X \approx 3{,}0$). Concomitantemente, a fila subiu de forma contínua a ponto da amostragem global aferir uma média estrondosa na casa das ≈ 866 requisições presas no sistema, algo previso unicamente pelos cálculos fluídicos de transição de fase de congestionamento.

G. Ponto Extra: Fila M/M/1/K com Capacidade Limitada

Como extensão do modelo, limitou-se a capacidade de cada servidor a K requisições simultâneas (fila + serviço). Requisições encaminhadas a um servidor já saturado são **descartadas** imediatamente, sem retry. Mantém-se a política

Aleatória e $\mu = 1{,}0$, de modo que cada servidor permanece um processo M/M/1/K independente com taxa efetiva de chegada $\lambda_i = \lambda/3$ e intensidade $\rho_i = \lambda_i/\mu$.

Para $\rho_i \neq 1$, as probabilidades de estado estacionário são

$$p_k = \frac{1-\rho_i}{1-\rho_i^{K+1}} \cdot \rho_i^k, \quad k = 0, 1, \ldots, K. \tag{11}$$

A probabilidade de perda (*blocking*) ocorre quando o servidor encontra-se cheio ($k = K$):

$$P_{\text{perda}} = p_K. \tag{12}$$

Pela conservação de fluxo, a vazão efetiva global deixa de igualar λ e passa a ser

$$X = \lambda\,(1 - P_{\text{perda}})\,. \tag{13}$$

Note que P_{perda} depende apenas de ρ_i e K; para $\lambda = 2{,}7$ obtém-se $\rho_i = 0{,}90$ em todos os valores de K testados.

V. Resultados e Discussão

A. Aderência do Modelo e Impactos Não Lineares

A comparação entre as curvas analíticas estabelecidas via decomposição multinomial ($E[R]_{\text{ana}}$) e a métrica empírica recolhida para o sorteio equiprovável, destacada parcialmente nos cálculos anteriores, demonstra um casamento quase perfeito (erros inferiores a 3%). O comportamento exponencial de $E[R]$ revela que operar balanceadores "ingênuos"em alta carga ($\rho \approx 0{,}9$) deteriora severamente a resiliência distribuída (Fig. 2).

Contudo, os ganhos com inteligência computacional, conforme discutidos analiticamente no Item 4.4, saltam substancialmente aos olhos sob análise de variância de fila. Para o cenário mais agressivo antes da instabilidade ($\lambda = 2{,}7$), o simples controle local do Round-Robin reduz em $\sim 35\%$ o atraso médio contra a aleatoriedade estrita. Se o *overhead* de checagem do estado permitir o uso do Join-the-Shortest-Queue, ocorre uma otimização imensa: diminuição contundente de aproximadamente 60% no retardo do tempo de resposta (10,3 u.t. na randomização para parcos $4{,}1$ u.t. com o JSQ, validado pelas médias listadas integralmente no Apêndice).

B. Buffer Finito: Comparação Analítica vs. Simulação

Para validar o modelo M/M/1/K, simulou-se o cenário de alta carga $\lambda = 2{,}7$ com $K \in \{5, 10, 20\}$ (política Aleatória, 10 réplicas). A Tabela I e a Fig. 4 confrontam P_{perda} e a vazão efetiva X previstas analiticamente com as médias empíricas.

Com $K = 5$, a perda analítica ($12{,}6\%$) subestima levemente a simulada ($14{,}6\%$), pois o descarte ocorre *após* o roteamento: uma chegada direcionada a servidor cheio é perdida mesmo que outro servidor tenha vaga. Para $K \geq 10$, analítico e simulado convergem (erros inferiores a 2% em X e P_{perda}). Em todos os casos, aumentar K reduz P_{perda} e aproxima X de $\lambda = 2{,}7$, mas nunca a atinge completamente enquanto $\rho_i = 0{,}90 < 1$ com buffer finito.

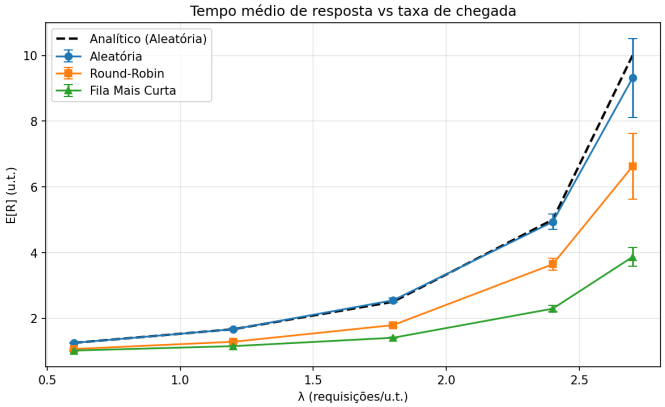


Figura 2. $E[R]$ analítico M/M/1 (Aleatória) e simulado em função da carga entrante λ para as três políticas investigadas.

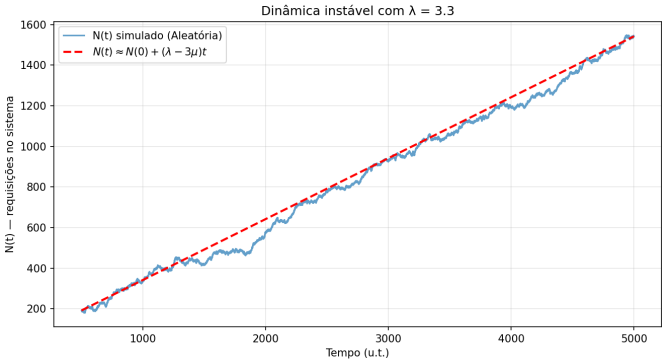


Figura 3. Evolução temporal $N(t)$ para a situação catastrófica com $\lambda = 3{,}3$. Observa-se a conformidade com a reta da equação fluida $N(0) + 0{,}3t$.

Tabela I
M/M/1/K por servidor ($\lambda = 2{,}7$, $\rho_i = 0{,}90$, política Aleatória).

K	$P_{\text{perda}}^{\text{ana}}$	$P_{\text{perda}}^{\text{sim}}$	X^{ana}	X^{sim}
5	0,126	0,146	2,360	2,358
10	0,051	0,050	2,563	2,557
20	0,014	0,012	2,663	2,654

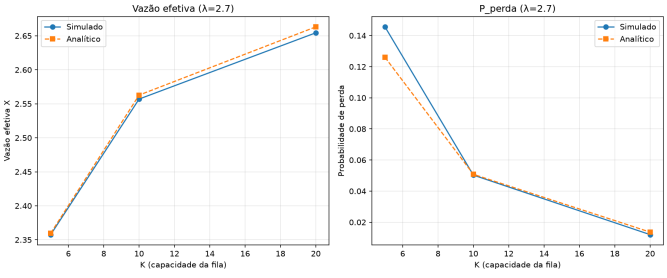


Figura 4. Vazão efetiva X e probabilidade de perda P_{perda} em função de K para $\lambda = 2{,}7$: simulação (média de 10 réplicas) vs. modelo M/M/1/K analítico.

VI. Conclusão

O desenvolvimento do simulador estocástico para balanceadores de carga serviu simultaneamente como ambiente de experimentação empírica e validador de hipóteses de modelagem analítica.

Através das meticulosas derivações que partem do teorema fundamental de *splitting* de Poisson, conseguiu-se construir um mapeamento acurado e matematicamente correto da matriz comportamental e probabilística na política de base (Aleatória). Os preceitos imutáveis da teoria clássica de filas, em especial a conservação de trabalho e o escalonamento invariante atestado pelo teorema de Little, foram provados não apenas na manipulação algébrica das equações fundamentais, como submetidos à verificação cega num motor computacional por eventos no limiar do tempo contínuo com aderência virtualmente milimétrica (erros percentuais na margem extrema de 10^{-4}).

Do ponto de vista prático, confirmou-se que políticas passivas, sem leitura do ecossistema computacional atrelado, criam severos gargalos estocásticos. A implementação informacional providenciada pelo *Join the Shortest Queue* evidenciou que a alocação determinística que ataca variâncias de filas em *real-time* salva parcelas vitais do *throughput* útil (tempo de latência), permitindo que arquiteturas lidem melhor com estresse perigosamente perto de saturações operacionais sem incorrer nos comportamentos colossais de divergências que invadem as predições fluidas lineares.

O ponto extra com buffers finitos M/M/1/K ($K \in \{5, 10, 20\}$) confirmou a relação $P_{\text{perda}} = p_K$ e $X = \lambda(1 - P_{\text{perda}})$, com aderência crescente entre teoria e simulação à medida que K aumenta. Trabalhos futuros incluem extensão a servidores heterogêneos (μ variáveis), políticas sensíveis a custos (Weighted Round-Robin) e modelagem analítica de roteamento com descarte pós-balanceamento.

Apêndice A
Tabelas Complementares

Nota: matrizes numéricas pós *warm-up* que balizam as Seções IV e V.

Tabela II
RESULTADOS DIRETOS DO MODELO ANALÍTICO M/M/1 ESTRITO PARA ALOCAÇÃO ALEATÓRIA ($\mu = 1{,}0$).

λ	ρ_i	U_i	$E[N_i]$	$E[R]$	Vazão X
0,6	0,20	0,20	0,25	1,25	0,60
1,2	0,40	0,40	0,67	1,67	1,20
1,8	0,60	0,60	1,50	2,50	1,80
2,4	0,80	0,80	4,00	5,00	2,40
2,7	0,90	0,90	9,00	10,00	2,70

Tabela III
CONSOLIDAÇÃO DOS DADOS CAPTURADOS NA SIMULAÇÃO DISCRETA (VALOR GERADO VIA 10 RÉPLICAS 5000 U.T.).

Política	Carga Injetada (λ)	Vazão Sistema (X)	Latência Total ($E[R]$)	População Total ($E[N]$)	Utilização U_{avg}	Referencial Analítico $E[R]_{\text{ana}}$
Aleatória	0,6	0,599	1,253	0,751	0,200	1,250
Aleatória	1,2	1,196	1,682	2,012	0,400	1,667
Aleatória	1,8	1,805	2,505	4,527	0,601	2,500
Aleatória	2,4	2,409	4,905	11,817	0,800	5,000
Aleatória	2,7	2,703	10,300	27,840	0,901	10,000
Round-Robin	0,6	0,603	1,057	0,637	0,200	1,250
Round-Robin	1,2	1,203	1,293	1,555	0,402	1,667
Round-Robin	1,8	1,803	1,829	3,298	0,603	2,500
Round-Robin	2,4	2,412	3,495	8,434	0,803	5,000
Round-Robin	2,7	2,694	6,461	17,408	0,897	10,000
JSQ	0,6	0,605	1,031	0,624	0,203	1,250
JSQ	1,2	1,198	1,139	1,365	0,400	1,667
JSQ	1,8	1,803	1,431	2,580	0,603	2,500
JSQ	2,4	2,404	2,286	5,497	0,801	5,000
JSQ	2,7	2,704	4,138	11,202	0,902	10,000

Tabela IV
VERIFICAÇÃO UNIVERSAL DA LEI DE LITTLE EM REGIMES TRANSIENTES E SUB-SATURAÇÃO ($E[N] = X \cdot E[R]$).

Comportamento Lógico	Entrada (λ)	Produto Analógico $X \cdot E[R]$	Integrado Simulado $E[N]$	Divergência Estocástica (%)
Aleatória	0,6	0,751	0,751	0,006
Aleatória	1,2	2,012	2,012	0,001
Aleatória	1,8	4,524	4,527	0,071
Aleatória	2,4	11,816	11,817	0,012
Aleatória	2,7	27,863	27,840	0,082
Round-Robin	0,6	0,637	0,637	0,022
Round-Robin	1,2	1,555	1,555	0,010
Round-Robin	1,8	3,299	3,298	0,014
Round-Robin	2,4	8,429	8,434	0,054
Round-Robin	2,7	17,408	17,408	0,000
JSQ	0,6	0,624	0,624	0,002
JSQ	1,2	1,365	1,365	0,013
JSQ	1,8	2,580	2,580	0,011
JSQ	2,4	5,498	5,497	0,010
JSQ	2,7	11,196	11,202	0,053